FAQ-20: How Do I Monitor Zscaler With Dynatrace?

> **Series:** FAQ — Frequently Asked Questions | **Reference:** 20 — Monitoring Zscaler With Dynatrace | **Created:** July 2026 | **Last Updated:** 11/13/2026

Overview

Zscaler sits in the path of everything. Every user reaching a SaaS application, every user reaching a private application, and — where ZDX is licensed — the experience of those users on their own devices. When it degrades, the symptom presents as "the application is slow," and the application team gets the ticket. That mismatch between where the problem is and where the ticket lands is the reason this integration keeps coming up.

The starting position in most organizations is worse than it needs to be. The three product lines write to three separate data planes with no shared view, and the practical consequence is a troubleshooting workflow built on exporting logs from the admin console and pivoting them in a spreadsheet. That workflow does not scale, does not correlate, and cannot answer the question anyone actually asked, which is whether the user's experience was bad and why.

Two things about this integration are worth knowing before you plan it, because both are commonly assumed the other way:

- **ZIA and ZPA have Dynatrace extensions; ZDX does not.** *Corrected 11/11/2026 — earlier revisions of this entry said ZPA had no extension, and before 09/28/2026 that the Hub listed no Zscaler integration at all.* A Dynatrace **Zscaler Internet Access (ZIA) extension** is on the Hub (minimum Dynatrace version 1.345): it takes the ZIA Cloud NSS feeds, extracts `zia.*` metrics, and discovers ZIA Location, User and Tunnel entities in Smartscape. A **Zscaler Private Access (ZPA) extension** (minimum Dynatrace version 1.345; docs published 10/09/2026, announced in the SaaS 1.348 release notes) does the same for ZPA Log Streaming Service (LSS) logs — `zpa.*` metrics and ZPA user, application and App Connector topology — but still needs a log shipper you run, because LSS cannot reach Dynatrace on its own. No ZDX extension or Hub listing was found (checked 11/11/2026), so for ZDX you are still building and owning the integration — a normal position, but one to decide deliberately rather than discover in week three. Re-check the Hub before planning; listings appear over time.

- **The three product lines do not share a transport.** ZIA can push over HTTPS; ZPA cannot; ZDX telemetry is pull-only — only ZDX *alerts* can be pushed, by webhook. A single ingestion design will not serve all three.

This entry is the **worked example of FAQ-19**. Every structural decision — route, retention, topology, isolation — is made there generically; here they are made concretely for Zscaler. Read FAQ-19 first if you want the reasoning; read this if you want the answers.

Table of Contents

1. Short Answer
2. Extensions for ZIA and ZPA, Build-Your-Own for ZDX
3. The Three Data Planes and What Each Answers
4. Transport Reality — Why One Route Will Not Work
5. Recommended Ingestion Architecture
6. What to Extract, What to Discard
7. Suggested Topology Model
8. Field Mapping Starting Point
9. Dashboards, Alerting, and SLOs
10. Synthetic Monitoring — When It Adds Anything
11. Implementation Sequence
12. Administrator Checklist
13. Recommended Approach
14. Common Gotchas

Prerequisites

Requirement	Details
Zscaler licensing	ZIA and/or ZPA for the log feeds. **ZDX is a separate SKU** — § 3 explains why its absence changes what the integration can deliver, and it is worth confirming before scoping. **ZDX API access requires the ZDX Advanced Plan** (§ 4.2)

Requirement	Details
Zscaler admin access	Rights to configure NSS / Cloud NSS feeds (ZIA), LSS receivers (ZPA), and to create ZDX API keys. Usually a network or security team, not the observability team
Dynatrace environment	SaaS with **Grail** and **OpenPipeline**
Network path	For ZPA LSS, a reachable TCP listener that App Connectors can egress to (§ 4)
Security sign-off	These feeds carry user identity, destination, device, and policy context. Decide retention before building — see FAQ-19 § 4 and § 6
Related reading	**FAQ-19** (the generic pattern this entry instantiates), **OPIPE** / **OPLOGS** (OpenPipeline depth), **ORGNZ** (buckets), **FAQ-15** (DPL, for parsing CSV/TSV feeds), **ALERT** / **SLO** / **DASH** (downstream)

---

1. Short Answer

Activate the Dynatrace ZIA extension for ZIA and the ZPA extension for ZPA — ZPA still reaches Dynatrace through a log shipper you run — pull ZDX through its API if you have it, extract metrics and events at ingest, discard the raw records by default, and model what the extensions do not cover — ZDX, or a ZPA pipeline you build yourself — as `CUSTOM_*` Smartscape nodes.

Product line	Transport it offers	Route into Dynatrace
ZIA (internet / SaaS)	Cloud NSS — HTTPS feed to an API-based collector	**Activate the ZIA extension** (Dynatrace 1.345+): Cloud NSS feeds push direct to the log ingest API, and the extension supplies parsing, `zia.*` metrics, topology and dashboards
ZPA (private access)	LSS — **raw TCP, optionally TLS**	**Activate the ZPA extension** (Dynatrace 1.345+) and run a **log shipper** between LSS and the Dynatrace ingest API — LSS cannot reach a Dynatrace API unaided. The extension supplies parsing, `zpa.*` metrics, topology and dashboards

Product line	Transport it offers	Route into Dynatrace
ZDX (digital experience)	**REST API, pull only** for scores and metrics (API key exchanged for a bearer token); alerts can also be pushed by webhook	**Scheduled poller** — workflow, extension, or collector

And the single most important scoping point, stated plainly:

> **ZIA and ZPA logs are not a substitute for ZDX.** They are the platform's own record of transactions it handled — excellent for access, policy, throughput, and connection outcomes. They do not contain device health, network-path quality, or page-fetch timing, because Zscaler does not observe those from the proxy. If the question is "why was it slow for this user," logs alone will not answer it, and no amount of ingestion engineering changes that.
>
> **Sources:** Zscaler Internet Access (ZIA) extension (DT docs) — *"Activation has three parts: activate the extension from Dynatrace Hub, configure the two OpenPipeline dynamic routes, and configure the Zscaler Internet Access Cloud NSS feeds."*, Zscaler Private Access (ZPA) extension (DT docs) — *"so a log shipper between ZPA and Dynatrace is required rather than optional"*, Integrating Cloud NSS with Cloud-Based SIEMs (Zscaler), Log Streaming Service (LSS) (Zscaler), Understanding the ZDX API (Zscaler Legacy APIs), About Webhooks (Zscaler).

---

2. Extensions for ZIA and ZPA, Build-Your-Own for ZDX

> **Correction (11/11/2026).** Earlier revisions of this section said ZPA had no Dynatrace extension or Hub listing. Dynatrace now publishes a **Zscaler Private Access (ZPA) extension** (docs published 10/09/2026, announced in the SaaS 1.348 release notes), alongside the **Zscaler Internet Access (ZIA) extension** recorded in the 09/28/2026 correction. It remains true for ZDX. The build-your-own guidance below still applies to ZDX, to any ZIA or ZPA feed type the extensions do not parse, and to a ZPA pipeline you choose to build yourself.

2.1 ZIA — activate the extension

The ZIA extension requires **Dynatrace version 1.345** or later and OpenPipeline Settings 2.0. Activation is three steps — activate it from the Hub, configure its two OpenPipeline dynamic routes (logs and metrics), and point the ZIA **Cloud NSS feeds** at the Dynatrace log ingest API with a `logs.ingest` token, using the extension's exact JSON feed output format (§ 6). In return you get:

What the extension supplies	Detail
Parsing	A log pipeline that recognizes each supported feed by `sourcetype`
Metrics	Counter and value metrics under the `zia.*` namespace
Topology	**ZIA Location**, **ZIA User** and **ZIA Tunnel** Smartscape entities and their relationships
Content	Per-log-type dashboards, unified analysis screens, and alert templates

It covers five Cloud NSS feed types — **Web, Firewall, DNS, Tunnel, and Admin Audit**. Any other ZIA feed you need (DLP, for instance) is back in build-your-own territory.

The extension changes three things elsewhere in this entry. Its metric namespace is `zia.*`, not the `zscaler.zia.*` scheme suggested in § 8 — do not build a parallel ZIA scheme. It models **users as entities** (ZIA User), which § 7 advises against for a hand-built model — keep your custom model to what the ZIA and ZPA extensions do not cover, and do not add a second user entity. And its topology uses its own node types, so the § 7 custom nodes are for what neither extension covers.

2.2 ZPA — activate the extension, and run a log shipper

The ZPA extension requires **Dynatrace version 1.345** or later and OpenPipeline Settings 2.0. Activation has five parts — activate it from the Hub, configure its two OpenPipeline dynamic routes (logs and metrics), deploy a log shipper, configure the ZPA LSS Log Receivers, and verify. In return you get:

What the extension supplies	Detail
Parsing	A log pipeline that recognizes each supported LSS log type by its `sourcetype` value
Metrics	Counter and value metrics under the `zpa.*` namespace

What the extension supplies	Detail
Topology	ZPA User, Application, App Group, App Connector and App Connector Group entities, plus Private Service Edge and Private Cloud Controller entities and their groups
Content	Dashboards, unified analysis screens, and alert templates — the templates ship disabled

It covers nine LSS log types — App Connector Metrics and Status, User Activity, User Status, Browser Access, and Private Service Edge and Private Cloud Controller Metrics and Status. Any other ZPA log type you stream is still ingested, but gets no packaged metrics, dashboards or topology.

What the extension does **not** remove is the transport constraint in § 4.1. It bundles no data source, and LSS still cannot post to a Dynatrace endpoint, so you deploy and operate a log shipper. Its page documents two — **Fluent Bit** (to the log ingest API) and **BindPlane** (to the OTLP logs endpoint) — and says any shipper capable of a TCP input and an HTTP output can fill the role. Three constraints from that page shape the rest of this entry:

- **Exact templates.** Each Log Receiver must use the extension's own JSON log stream template. The ZPA default template cannot be used, because it carries no field identifying the log type.
- **Unmodified records.** The shipper must forward each record exactly as LSS wrote it; one that reformats, enriches, truncates, reorders or drops part of a record breaks the parsers. The page's way to reduce volume is to trim each template to the fields the extension parses — not to edit records in the shipper.
- **One endpoint is enough.** Every log type can share one shipper endpoint, because each record carries its own `sourcetype`. You still configure one Log Receiver per log type.

The extension changes three things elsewhere in this entry. Its metric namespace is `zpa.*`, not the `zscaler.zpa.*` scheme suggested in § 8. Its topology uses its own ZPA node types, which cover the ZPA side of the § 7 custom model. And it models **users as entities** (ZPA User). The hand-built ZPA guidance in §§ 4–8 remains as the alternative for a pipeline you build yourself; where you use the extension, its page is the documented route.

2.3 ZDX — no listing, so you own it

No ZDX extension or Hub listing was found (checked 11/11/2026). The ingestion primitives you need all exist, but the ownership shape of the project is different, and it is better acknowledged at planning time:

What "no listing" means	Consequence
You own the parsing	Field extraction is yours to write and maintain. There is no vendor-maintained mapping that updates itself
You own the schema drift	When Zscaler adds or renames a log field, nothing warns you. Your pipeline quietly produces nulls
You own the failure modes	An expired ZDX credential surfaces as missing data, not as an alert — unless you build that alert
Support boundaries are yours to navigate	Dynatrace supports its ingest APIs and OpenPipeline; Zscaler supports its feeds. The integration between them is yours
Effort is front-loaded but bounded	Once field mapping and topology are settled, the pipeline is stable. The cost is a project, not a permanent tax

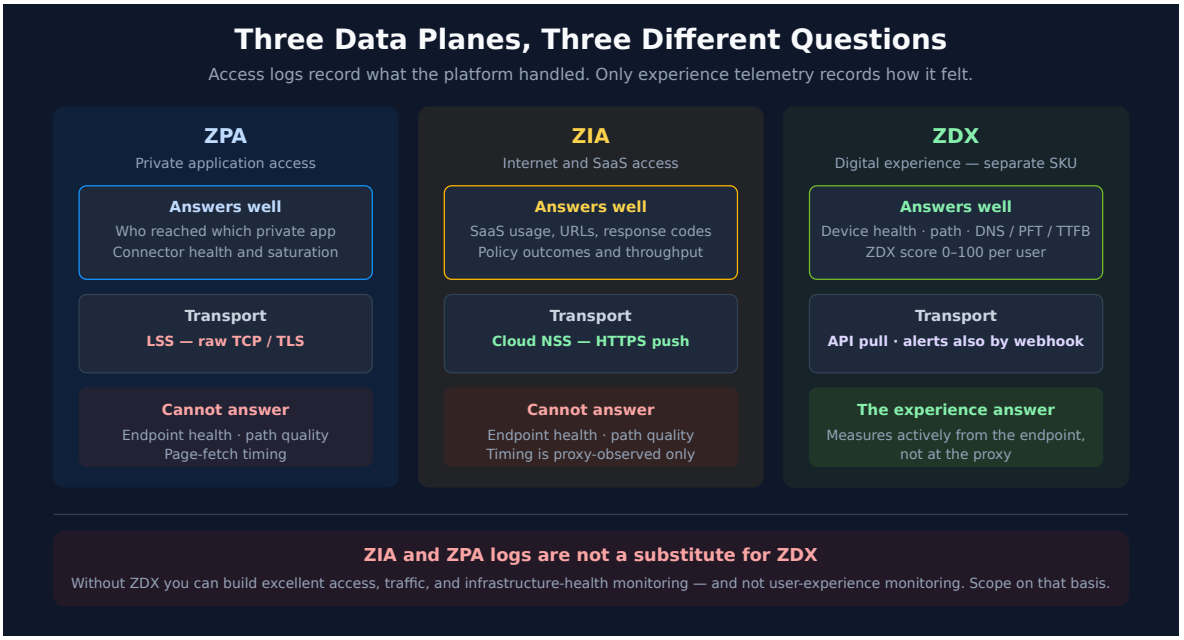
The one thing worth doing before you start: re-check the Hub. The ZIA and ZPA extensions are the proof that vendor integrations appear over time, and adopting a supported path is strictly better than maintaining a custom one.

> **Sources:**
>
> - Zscaler Internet Access (ZIA) extension (DT docs) — *"Minimum required Dynatrace version: Dynatrace version 1.345"*; discovers *"ZIA Location, ZIA User, and ZIA Tunnel entities"*
> - Zscaler Internet Access (ZIA) (Dynatrace Hub)
> - Zscaler Private Access (ZPA) extension (DT docs) — *"Minimum required Dynatrace version: Dynatrace version 1.345."*; *"The extension bundles no data source, so there is nothing to install on an ActiveGate."*; *"The custom log stream templates replace the ZPA defaults"*; *"The shipper must forward records unmodified"*; *"Every log type can share the same shipper endpoint"*
> - Zscaler Private Access (ZPA) (Dynatrace Hub)
> - SaaS 1.348 release notes (DT docs) — *"New Zscaler Private Access (ZPA) and Zscaler Internet Access (ZIA) extensions"*
> - Dynatrace Hub (Dynatrace) — no ZDX listing found, checked 11/11/2026
> - **Derived:** the ownership consequences in § 2.3 follow from the absence of a vendor-maintained listing, not from a Dynatrace statement about Zscaler specifically

---

3. The Three Data Planes and What Each Answers

Map your questions to data planes before configuring a single feed. This is FAQ-19 § 2 applied to Zscaler, and it is the step that prevents an expensive log-volume project that answers the wrong questions.



3.1 ZPA — private application access

Treat ZPA logs as a **session and access-path dataset**, not as generic log noise. They describe how users reach private applications through the ZPA fabric: identity, application, application group, service edge, App Connector, connection status, byte counts, and timing fields where present.

- **User Activity** (`zpn_trans_log`) — who reached which private application, from where, and whether the session succeeded.
- **App Connector Status and Metrics** (`zpn_ast_auth_log` , `zpn_ast_comprehensive_stats`) — whether connector infrastructure is healthy, saturated, disconnected, or unbalanced. This is the highest-value ZPA feed for infrastructure alerting and the one most often overlooked.
- **Timing fields** are **Zscaler-observed latency indicators**, not endpoint experience. Connection-setup time is a real, useful signal about the access path; it is not page-load time and should never be presented as such.

3.2 ZIA — internet and SaaS access

ZIA provides web, DNS, firewall, tunnel, DLP, and SaaS traffic metadata — proxy-observed visibility into usage, destinations, ports, categories, policy outcomes, errors, and throughput.

- **Web logs** — SaaS application access, URLs, response codes, application names and categories, request and response sizes.
- **DNS and firewall logs** — destination resolution, ports, protocols, allow/deny behavior, traffic patterns.
- **Proxy-level performance indicators** can be derived, but they are measured at the proxy. Present them as such.

3.3 ZDX — digital experience

ZDX is the only one of the three that closes the experience gap, because it measures actively from the endpoint rather than recording transactions at the proxy. It produces a **ZDX Score per user from 0–100**, aggregated over user, application, location, department, and organization, plus:

- **Device layer** — CPU, memory, Wi-Fi quality, device health, endpoint events.
- **Network layer** — latency, packet loss, hop and path visibility, ISP and last-mile indicators, cloud-path probe output.
- **Application layer** — DNS time, page-fetch time, server response / TTFB, availability.

ZDX covers customer-defined applications alongside predefined ones such as Microsoft 365, Salesforce, ServiceNow, Zoom, and Box.

> **Sources:**
>
> - Log Streaming Service (LSS) (Zscaler) — ZPA log types and the raw TCP/TLS receiver transport
> - Integrating Cloud NSS with Cloud-Based SIEMs (Zscaler) — the ZIA HTTPS push feed
> - Understanding the ZDX API (Zscaler Legacy APIs) — the API pull and the two-hour report window; About Webhooks (Zscaler) — the alert push path
> - Understanding the ZDX Score (Zscaler) — *"The ZDX Score is based on a scale of 0 (lowest) to 100 (highest)"*, with scores for users, applications, departments, locations and cities
> - **Derived:** the *cannot answer* column is this entry's reading of what each plane's vantage point excludes — no Zscaler page states the exclusions as such.

3.4 Which plane answers which question

Question	ZPA logs	ZIA logs	ZDX	How to combine them
Who accessed what?	Strong — private apps	Strong — SaaS and web	User and app context	Normalize `user` / `app` into shared dimensions; hash `user` (§ 6)
Was it slow?	Connection-setup timing proxies	Proxy-level transaction timing	**Best source**	Use logs to localize, ZDX to characterize experience
Is the endpoint unhealthy?	No	Limited / indirect	**Yes**	ZDX device metrics only
Is an ISP or network path degraded?	Limited	Limited	**Yes**	ZDX cloud-path metrics only
Is connector infrastructure saturated?	**Yes** — connector metrics and status	No	Indirect, via app experience	Model connectors as entities (§ 7); alert on saturation and disconnects
Can raw records be minimized?	Yes	Yes	Pull is already aggregated	Extract metric / event / alert, then discard (§ 6)

The honest conclusion. Without ZDX, you can build a genuinely good access, traffic, and infrastructure-health integration — and you cannot build user-experience monitoring. Scope the project on that basis rather than discovering it at the demo.

> **Sources:**
>
> - Understanding User Activity Log Fields (Zscaler)
> - zpa_lss_config_controller (Zscaler GitHub) — the `zpn_*` log-type codes cited in § 3.1
> - Understanding the ZDX Score (Zscaler) — the 0–100 ZDX Score for users, applications, departments, locations and cities
> - Zscaler Digital Experience reference architecture (Zscaler)

---

4. Transport Reality — Why One Route Will Not Work

This is the section that most often invalidates a diagram drawn before the transports were checked. A typical first architecture shows "Zscaler → Dynatrace ingest" as a single arrow covering both ZIA and ZPA. **That arrow is correct for ZIA and wrong for ZPA**, and the difference is not a configuration detail — it decides whether you need to run infrastructure.

	ZIA — Cloud NSS	ZPA — LSS	ZDX — API
Direction	Push	Push	**Pull** for telemetry; alerts can also push by webhook
Transport	**HTTPS** to an API-based collector	**Raw TCP**, optional TLS	HTTPS REST
Auth	Custom headers on the feed	None at transport level — network reachability is the control; optional mutual TLS	**API key** (key ID + secret) exchanged at `/oauth/token` for a 60-minute bearer token
Formats	**JSON** — the exact format the ZIA extension requires; custom templates only for a self-built pipeline	CSV, TSV or **JSON** templates — the ZPA extension requires its own JSON templates	JSON
Reaches Dynatrace unaided?	**Yes**	**No — collector hop required**	No — needs a scheduled caller

4.1 ZPA LSS — the constraint to design around

LSS is configured as a **log receiver** defined by a host, a port, and an optional TLS flag; App Connectors open a TCP connection to it and stream. There is no HTTPS output and no custom-header support, which means **LSS cannot authenticate to the Dynatrace log ingest API**. Something must terminate the TCP stream and forward it.

> **Two pages describe the direction differently.** Zscaler describes LSS as a service that *"initiates a log stream through a Public Service Edge for Private Access"*, and says the App Connector *"receives the log stream and then forwards it to a log receiver"*. On that reading, the connection your receiver accepts comes from an App Connector in your own network, which is how this section describes it. The Dynatrace ZPA extension page (published 10/09/2026) says, about testing the inbound path, that *"LSS connects from the*

> *Zscaler cloud and there is no host there to run a command on."* This entry follows Zscaler, which documents its own data path. If your receiver admits only known sources, check at the shipper which addresses actually connect before you finalize firewall rules.

Your options:

Terminator	When it fits
Fluent Bit or BindPlane	The two shippers the ZPA extension documents (§ 2.2). Fluent Bit posts to the log ingest API, BindPlane to the OTLP logs endpoint; both forward records unparsed
OpenTelemetry Collector	The general answer. Run it near the connectors, use a TCP or syslog receiver, forward to Dynatrace
ActiveGate syslog ingestion	Self-built pipelines only, if you template LSS output to syslog format — the ZPA extension needs its own JSON templates. Environment ActiveGate on Linux, 1.295+; multi-environment ActiveGates are not supported
An existing pipeline product	If Zscaler logs already flow to a SIEM through one, multi-home an output (§ 5.2) — usually the least new infrastructure

Three further LSS properties shape the build:

- **One log type per receiver configuration.** User Activity, App Connector Metrics, and App Connector Status are three separate LSS configurations. Each names its own host and TCP port. With the ZPA extension's templates every log type can share one shipper endpoint, because each record carries its own `sourcetype` (§ 2.2). For a self-built pipeline on a template without a log-type field, community practice is one port per log type, so the collector can tell the streams apart. Plan the configuration count up front.
- **Choose JSON as the log template.** With the ZPA extension that means its own published templates, which are required (§ 2.2). For a self-built pipeline, LSS offers CSV, TSV, and JSON templates. CSV means positional DPL parsing that breaks silently when Zscaler adds a column, whereas JSON tolerates schema growth.
- **A receiver outage is data loss.** Apart from audit logs, LSS does not send what was generated while the connection to the receiver was down. Run the collector highly available if gaps in the feed matter to you.

4.2 ZDX — pull for telemetry, webhooks for alerts

The ZDX API authenticates with an **API key** — a key ID and secret created on the ZDX Admin Portal — exchanged at `/oauth/token` for a bearer token that lasts 60 minutes. API access requires the **ZDX Advanced Plan**; confirm it in phase 1. Zscaler files this flow under *Legacy Zscaler APIs*, and its getting-started page sends anyone using Zscaler OneAPI endpoints to

API clients in the Authentication Service instead — check which applies to your tenant before you build the poller. The property that shapes your poller: most report endpoints serve a **two-hour window per request**, so longer ranges require multiple calls.

ZDX **alerts** have a second route: ZDX can push them by webhook, with Basic, Token, or OAuth authentication. That is a push path for alert notifications, not for scores or device metrics. Zscaler's webhook page does not say whether a ZDX webhook can call the Dynatrace ingest API directly with the authorization header it requires — test that, or plan a small relay, before relying on it.

Practical consequences:

- **Poll on a schedule near the window**, not aggressively. A poller asking for two hours every two hours is the natural cadence; asking every minute mostly re-fetches.
- **Lag the window.** Zscaler estimates a 20-minute delay from collection to reporting, so a window fetched the moment it closes is incomplete. Offset each request by that delay, or re-fetch the previous window.
- **Drop `-1` scores before ingest.** A ZDX Score of `-1` means no data is available, not a terrible score — ingested as a value it drags every average down and fires score-degradation alerts.
- **Backfill is a loop, not a parameter.** Seeding history means iterating windows — build that in rather than bolting it on.
- **Credential expiry is a silent failure.** The bearer token lasts an hour, so the poller re-authenticates on every run; a revoked or expired API key stops it. Nothing in Dynatrace knows the ZDX poller stopped. Alert on the *absence* of the ZDX metrics you extract, not just on their values.

> **Sources:**
>
> - Log Streaming Service (LSS) (Zscaler) — *"initiates a log stream through a Public Service Edge for Private Access"*; *"With the exception of audit log data, the LSS does not transmit any log data generated during a connection loss between the App Connector and the SIEM."*
> - Zscaler Private Access (ZPA) extension (DT docs) — *"LSS connects from the Zscaler cloud"*, read 11/11/2026
> - Configuring a Log Receiver (Zscaler) — one Log Type, one TCP port and one Log Template (CSV, TSV or JSON) per receiver configuration
> - Understanding the ZDX API (Zscaler Legacy APIs) — *"Most Report API endpoints require a 2-hour time range to provide 2 hours of data."*; *"This can cause a delay from collection to reporting, which is estimated to be 20 minutes."*; *"If you receive a ZDX Score of -1 on the ZDX API, then there is no data available."*
> - Getting Started with the ZDX API (Zscaler Legacy APIs) — *"Be a ZDX Advanced Plan User."*; *"Upon successful authentication, a bearer token will last 3,600 seconds or 60 minutes."*

- About Webhooks (Zscaler) — *"When an event occurs, the webhook sends real-time data to a specific URL or API endpoint as an HTTP callback."*
- Integrating Cloud NSS with Cloud-Based SIEMs (Zscaler)
- Zscaler Internet Access (ZIA) extension (DT docs) — *"Each feed must use the exact JSON Feed Output Format below so that field names match the pipeline parsers."*
- Syslog ingestion with ActiveGate (DT docs)
- Zscaler Private Access (ZPA) extension (DT docs) — *"so a log shipper between ZPA and Dynatrace is required rather than optional"*; *"Every log type can share the same shipper endpoint"*
- **Derived:** the ActiveGate-syslog row is limited to self-built pipelines because the ZPA extension needs its own JSON templates, forwarded unmodified. The one-port-per-log-type layout is community practice for self-built pipelines, not a documented requirement

---

5. Recommended Ingestion Architecture

Two patterns, chosen by whether Zscaler logs already flow through a pipeline product you operate.



5.1 Pattern A — direct, plus a log shipper for ZPA

Appropriate when Zscaler logs are not already centralized, or when the existing pipeline is not something you want to depend on.

Feed	Path
ZIA	Cloud NSS → **Dynatrace log ingest API** → the **ZIA extension's** OpenPipeline routes (§ 2.1)
ZPA	LSS → **log shipper** (Fluent Bit, BindPlane or an OTel Collector) → Dynatrace → the **ZPA extension's** OpenPipeline routes (§ 2.2). ActiveGate syslog only for a self-built pipeline
ZDX	Scheduled poller → Dynatrace metric ingest → OpenPipeline

You run and monitor one log shipper — highly available if gaps matter, because LSS does not resend what it generated while the receiver was unreachable (§ 4.1). In exchange, the observability feed has no dependency on another team's platform.

5.2 Pattern B — multi-home from an existing pipeline

Appropriate — and usually preferable — when Zscaler logs already reach a SIEM or security data lake through a pipeline product. **Add an output; do not re-point the source.**

Feed	Path
ZIA + ZPA	Zscaler → existing pipeline → *(existing SIEM output, untouched)* + **new Dynatrace output**. For ZIA, the extension's route matches on the NSS `sourcetype`, so the forwarded records must keep the Cloud NSS JSON shape it expects — or send ZIA direct (Pattern A) and multi-home ZPA only. For ZPA under the extension, the pipeline must forward its LSS records unmodified (§ 2.2)
ZDX	Scheduled poller → Dynatrace (unchanged — ZDX is pull, so the pipeline is not in this path)

Why this is usually the right enterprise answer:

- **The security pipeline is undisturbed.** Re-pointing a feed a SOC depends on is a change with a blast radius outside observability. Adding an output is not.
- **You get a transformation point you control.** Mask, hash, and reduce fields on the observability copy while the security copy stays complete — except on records an extension parses, which must keep the extension's exact format (§ 2.2, § 6).
- **The ZPA transport problem is already solved.** The pipeline terminates TCP already; its outputs are ordinary HTTP. No new collector.

Dynatrace publishes Hub destinations for at least one common pipeline product over both HTTP and OpenTelemetry, so this is a supported path rather than an improvisation.

The trade-off, stated plainly: the observability feed inherits the pipeline's availability, release cadence, and change process. Where that pipeline is already business-critical, this costs nothing. Where it is a best-effort side system, Pattern A is more honest.

Note that ZDX telemetry does not participate in either pattern's log path. It is an API pull producing metrics, and it stays a separate track. Teams routinely under-scope it because the architecture diagram shows one Zscaler box.

> **Sources:** Zscaler Private Access (ZPA) extension (DT docs) — *"The shipper must forward records unmodified"*, Cribl via HTTP (Dynatrace Hub), Cribl via OpenTelemetry (Dynatrace Hub), Syslog via OpenTelemetry Collector (Dynatrace Hub).

---

6. What to Extract, What to Discard

Zscaler logs carry user identity, destination, device, policy, and application context. Raw retention should be a decision someone signs, not a default someone inherits. The mechanics — and the `Drop record` versus `No storage assignment` trap that silently defeats this pattern — are in **FAQ-19 § 4**; do not configure this without reading it.

Extract	Example	Retention posture
Metric	Average ZPA connection-setup time by application; ZIA request and error counts by application; connector CPU / memory / active connections; ZDX score by application and location	**Keep the metric.** Discard the source record unless separately justified
Business event	A user reached a critical private application from an unusual region	**Keep**, with minimal fields
Alert / problem	Connector disconnected; SaaS error rate spike; ZDX score degradation	**Keep** with entity context attached (§ 7)
Raw record	Full ZIA or ZPA transaction payload	**Discard by default.** Retain only where security or compliance asks — then in a dedicated bucket with its own permissions

Field-level controls in the Processing stage, before storage:

Field class	Control
`Username` and user identifiers	**Hash** — preserves correlation across ZIA / ZPA / ZDX without exposing identity. Note that a hash of a username drawn from a known directory is enumerable; this is a control against casual exposure, not a privacy guarantee

Field class	Control
Full URLs with query strings	**Mask** or truncate to host and path — query strings routinely carry tokens and identifiers
Fields with no analytical use	**Remove.** Cheapest control; apply first

Normalize before you extract. Zscaler's feeds disagree with each other about field names — `appname` in ZIA web logs, `Application` in ZPA User Activity, an application field again in the ZDX API response. Rename all three into a single canonical `app` dimension in the Processing stage. Metric dimensions and Smartscape ID components are computed from fields as they stand at their stage, so renaming afterwards leaves metrics dimensioned on the old names with no way to retrofit them.

ZIA under the extension. The extension parses only feeds that use its exact JSON output format: a changed `sourcetype`, a non-JSON output type, or renamed fields mean the logs are still ingested but not parsed — no metrics, no topology, empty dashboards. Its page documents parsing, metric extraction and topology, and says nothing about where the raw ZIA records are stored, so check where they land (the bucket query below shows it) rather than assuming they are discarded.

ZPA under the extension. The same holds for ZPA: records must use the extension's templates and arrive unmodified, or they are ingested but not parsed. Its page documents parsing, metrics and topology and does not say where the raw ZPA records are stored, so check that with the bucket query below as well.

If you run a **self-built** ZIA pipeline on a non-JSON NSS template instead, a parsing example is worth having in hand. This runs as-is and scans nothing:

> **Sources:** Processing in OpenPipeline (DT docs) — stage order, and the `Drop record` versus `No storage assignment` behaviour this table depends on, Configure data storage and retention for logs (DT docs) — route-based storage assignment and per-bucket retention. Zscaler Internet Access (ZIA) extension (DT docs) — *"an output type other than JSON, or renamed or missing fields mean the logs are still ingested but are not parsed."* Zscaler Private Access (ZPA) extension (DT docs) — *"a renamed field, or a missing one means the logs are still ingested but are not parsed"*. **Derived:** the retention-posture column applies FAQ-19 § 4's extract-then-discard rule to Zscaler's signal mix.

```
// DPL parse of a templated ZIA-style web log line — for a SELF-BUILT ZIA
// pipeline only; the ZIA extension requires its exact JSON feed format.
// Runs standalone against a synthetic record — zero bytes scanned.
// Adapt the matchers to your own NSS feed template.
// The leading LD skips the timestamp: in OpenPipeline the event timestamp is
// mapped by its own processor rather than parsed out of the message body.
data record(content = "2026-07-31T10:15:02Z user=jdoe@example.com app=Salesforce
url=https://example.my.salesforce.com/services/data respcode=200 reqsize=812
respsize=44311 location=US-East")
| parse content, "LD 'user=' STRING:user ' app=' LD:app ' url=' LD:url '
respcode=' INT:respcode ' reqsize=' INT:reqsize ' respsize=' INT:respsize '
location=' LD:location"
| fields user, app, respcode, reqsize, respsize, location
```

For a **self-built** ZPA pipeline on JSON output — recommended in § 4.1 — parsing is simpler and survives Zscaler adding fields. (With the ZPA extension, its own pipeline parses its templates.)

```
// JSON-format ZPA User Activity record, for a SELF-BUILT ZPA pipeline only —
// the ZPA extension parses its own templates. Also runs standalone at zero scan.
// Prefer this over CSV: adding a field upstream does not break the parse.
// Zscaler documents most ZPA timing fields in microseconds; the unit of
// ConnectionSetupTime is not stated — confirm it from your own feed before
// converting or labelling it.
data record(content = "{\"LogTimestamp\":\"2026-07-
31T10:15:02Z\",\"Username\":\"jdoe@example.com\",\"Application\":\"payroll-
app\",\"Connector\":\"dc1-connector-01\",\"ClientZEN\":\"iad3-zpa-
edge\",\"ConnectionStatus\":\"active\",\"ConnectionSetupTime\":42}")
| parse content, "JSON:j"
| fieldsAdd user = j[Username], app = j[Application], connector = j[Connector],
edge = j[ClientZEN], setup_time = j[ConnectionSetupTime]
| fields user, app, connector, edge, setup_time
```

While raw records are still being retained — during build-out, or where compliance requires it — confirm they land in the dedicated bucket rather than the default. This applies to the ZIA and ZPA extensions' records too. (Executed 10/02/2026: valid, no notifications, no rows on a tenant without Zscaler data. Re-run 10/15/2026: no rows, 0.11 GB scanned for the hour against 2.1 GB unfiltered. Re-run 11/11/2026 with the ZPA match added: valid, no notifications, no rows, 0.08 GB scanned for the hour.)

```
// Where are retained Zscaler records landing? Any row whose bucket is
// default_logs is a Zscaler record that missed its bucket assignment.
// ZIA feeds carry sourcetype "zscalernss-*" (the ZIA extension's route key);
// ZPA records sent with the ZPA extension's templates carry "zscalerlss" in
// content (its route key). The log.source match is a placeholder for however
// a self-built ZPA collector tags its records — adapt it. Every record in the
window is evaluated, and the
// bytes scanned follow what the filter matches (see FAQ-09): keep the window
short.
fetch logs, from:-1h
| filter matchesPhrase(sourcetype, "zscalernss") or matchesPhrase(content,
"zscalerlss") or matchesValue(log.source, "*zscaler*")
| summarize records = count(), by:{dt.system.bucket}
```

Once metric extraction is live, confirm the metrics exist before building dashboards on them. This costs nothing to run:

```
// Which Zscaler metrics has extraction actually produced?
// zia.* comes from the ZIA extension and zpa.* from the ZPA extension;
// zscaler.* is the self-built scheme suggested in section 8 — adjust it to
// whatever you settled on.
// Zero rows means extraction is not producing — check the pipeline before
// assuming a dashboard problem.
metrics
| filter startsWith(metric.key, "zia.") or startsWith(metric.key, "zpa.") or
startsWith(metric.key, "zscaler.")
| fields metric.key
```

And a representative consumption query over an extracted metric:

```
// Top SaaS applications by request volume, from the extracted metric
// rather than from a recurring log scan. zscaler.zia.request.count is the
// section 8 scheme for a SELF-BUILT ZIA pipeline; with the ZIA extension,
// query its zia.* metrics instead (list them with the query above).
timeseries requests = sum(zscaler.zia.request.count), from:-24h, by:{app,
location}
| fieldsAdd total = arraySum(requests)
| sort total desc
| limit 10
```

---

7. Suggested Topology Model

The highest-leverage design decision in this integration, and the one that pays off in every dashboard and alert afterwards. **Scope it to what the extensions do not cover** — the ZIA extension already discovers ZIA Location, ZIA User and ZIA Tunnel entities (§ 2.1), and the ZPA extension ZPA User, Application, App Group, App Connector and App Connector Group entities among others (§ 2.2). With both extensions in place the model below is for ZDX; its ZPA rows apply only to a ZPA pipeline you build yourself — do not build `CUSTOM_*` ZPA nodes alongside the extension's own. The mechanics — Smartscape ID components, `CUSTOM_` / `EXT_` naming, static versus dynamic edges, and the prerequisite that both IDs exist on the record before an edge processor runs — are in **FAQ-19 § 5**. What follows is the Zscaler-specific model.

Node type	Examples	Defined by	Why it earns entity status
`CUSTOM_SECURE_ACCESS_APP`	Salesforce, ServiceNow, Microsoft 365, `payroll-app`	ZPA User Activity (ZIA web logs only if you run your own ZIA pipeline instead of the extension)	The join key across the planes you model yourself. Routes metrics and alerts to the right application owner. With the ZPA extension in place, a ZDX record can meet the extension's `ZPA_APPLICATION` entity only on the application name: the extension page lists its node types but not how their IDs are built, so a hand-built edge to them cannot be specified
`CUSTOM_APP_CONNECTOR`	`dc1-connector-01`	ZPA App Connector Metrics	Enables connector health and saturation alerting — the clearest infrastructure win in the whole integration
`CUSTOM_CONNECTOR_GROUP`	`finance-app-connectors`	ZPA connector metadata	Regional and application-specific operations; the natural grouping for on-call ownership
`CUSTOM_SERVICE_EDGE`	`iad3-zpa-edge`	ZPA User Activity (`ClientZEN`)	Isolates edge-specific degradation from application-specific degradation

Node type	Examples	Defined by	Why it earns entity status
`CUSTOM_ACCESS_LOCATION`	`US-East`, a named office	ZPA and ZDX (ZIA locations are the extension's ZIA Location entity)	The primary triage and grouping axis, and the one executives ask about

Modeling notes that matter in practice:

- **One feed defines each node; the rest enrich by ID only.** App Connector Metrics defines `CUSTOM_APP_CONNECTOR` — User Activity references it. Extracting the same node from two feeds gives you two sources of truth for its fields.
- **Choose stable ID components.** Prefer Zscaler-assigned identifiers over display names. A connector renamed in the admin console should not become a second entity.
- **Static edges for structure, dynamic for observation.** A connector *belongs to* a connector group — static. A service edge *served* an application during a window — dynamic, which means both Smartscape IDs must already be on the record when the edge processor runs.
- **Do not model users as entities in your own model.** User identity is a hashed dimension (§ 6), not a node type. Modeling it produces enormous cardinality and a compliance surface you spent § 6 avoiding. The ZIA and ZPA extensions make the opposite choice — they discover ZIA User and ZPA User entities — so decide deliberately whether that is acceptable for your data, and do not add a second user entity for ZDX.
- **Resist over-modeling.** Five node types is a maintainable topology. Fifteen is a second system.

Verify the model exists before building on it — `smartscapeNodes` returns zero rows rather than an error for a type that was never extracted, so an empty result proves only that the query is valid:

> **Sources:** Zscaler Internet Access (ZIA) extension (DT docs) — the extension's ZIA Location / User / Tunnel entities, Zscaler Private Access (ZPA) extension (DT docs) — *"The extension creates ZPA_USER, ZPA_APPLICATION, ZPA_APP_GROUP"* (no ID components documented, checked 11/11/2026), Smartscape node and edge extraction in OpenPipeline (DT docs) — `CUSTOM_*` node and edge extraction, ID components, and the requirement that both IDs be present on the record before an edge processor runs, Define custom topology via OpenPipeline (DT docs) — end-to-end worked example. **Derived:** the five-node Zscaler model is this entry's proposal, not a documented reference architecture.

```
// Which self-built (CUSTOM_*) Zscaler node types has extraction produced?
// The ZPA extension's own node types are ZPA_* types, not CUSTOM_* (section 2.2).
// Zero rows before the pipeline runs is expected, not an error.
smartscapeNodes "CUSTOM_*", from:-7d
| dedup type
| fields type
```

```
// Stale App Connectors — the disconnect signal worth alerting on.
// Self-built model (section 7). With the ZPA extension, App Connectors are its
// own ZPA nodes, and its "Not Reporting" alert templates cover silence.
// lifetime[end] is a rolling "observed until" timestamp. A connector whose
// lifetime[end] is older than 10 minutes has stopped reporting.
//
// from:-7d is REQUIRED. Without it, smartscapeNodes returns only nodes seen
// in the default 2 h window, so a connector silent for longer is not returned
// at all and this check reads "none stale" exactly when connectors are lost.
// Verified 10/02/2026 on HOST: 6 stale hosts with from:-7d, 0 without it.
//
// lifetime[end] is ALWAYS populated, even on a stale node, so it is not a
// tombstone: null-testing it selects nothing. Compare against now(). Flip the
// comparison (&gt; now() - 10m) to list live connectors instead.
smartscapeNodes CUSTOM_APP_CONNECTOR, from:-7d
| fieldsAdd last_seen = lifetime[end]
| filter last_seen &lt; now() - 10m
| fields id, name, last_seen
```

```
// Which connectors serve which private applications — self-built model (section
7).
// Node types uppercase, edge types lowercase.
//
// traverse REPLACES the record with the target node's own fields — id, name,
type.
// There is no sourceName or targetName: DQL resolves unknown identifiers to null
// WITHOUT raising FIELD_DOES_NOT_EXIST, so selecting them returns the right
number
// of rows with every value empty and no error to tell you (verified 08/27/2026).
// A wrong edge type likewise returns zero rows with no error — check
smartscapeEdges
// if this comes back empty.
smartscapeNodes CUSTOM_SECURE_ACCESS_APP
| traverse edgeTypes: {served_by}, targetTypes: {CUSTOM_APP_CONNECTOR}, direction:
forward
| fields id, name, type
```

8. Field Mapping Starting Point

A starting point, **not a specification.** Zscaler field names vary by product line, feed type, NSS template, and account configuration. Validate every one against your own feed output before building on it — and expect at least a few to differ.

Source	Field class	Example source fields	Canonical Dynatrace dimension	Purpose
ZPA User Activity	Identity / app	`Username`, `Application`, `AppGroup`	`user` (hashed), `app`, `app_group`	Access analytics; `app` is the cross-plane join key — by name only where an extension owns the entity (§ 7)
ZPA User Activity	Path	`ClientZEN`, `Connector`, `ClientCity`	`service_edge`, `connector`, `location`	Root-cause isolation; feeds the § 7 ID components
ZPA User Activity	Timing	`ConnectionSetupTime`, `CAProcessingTime`, `AppLearnTime`	`setup_time` (unit not documented — confirm from your feed), `ca_time_us`, `app_learn_time_us` (documented in microseconds)	Latency **proxies** — Zscaler-observed, not endpoint experience
ZPA Connector Metrics	Capacity	`CPUUtilization`, `Memory`, `ActiveConnections`	`connector.cpu`, `connector.memory`, `connector.connections`	Connector saturation alerting
ZIA Web	SaaS / app	`appname`, `url`, `respcode`, `reqsize`, `respsize`	`app`, `url`, `response_code`, `bytes_in`, `bytes_out`	SaaS performance and error rates

Source	Field class	Example source fields	Canonical Dynatrace dimension	Purpose
ZDX API	Experience	score, DNS time, page-fetch time, TTFB, latency, packet loss	`zscaler.zdx.*` metrics	True user-experience monitoring

A suggested metric naming scheme. Consistency here is what makes dashboards and SLOs composable later. It is for ZDX and for any ZPA or ZIA pipeline you build yourself; **the ZIA extension's metrics are under `zia.*` and the ZPA extension's under `zpa.*`**, so the `zscaler.zia.*` and `zscaler.zpa.*` rows below apply only to pipelines you build yourself:

Signal	Metric key	Dimensions
ZDX score	`zscaler.zdx.score`	`app`, `user_group`, `location`
Page-fetch time	`zscaler.zdx.page_fetch_time`	`app`, `location`
DNS time	`zscaler.zdx.dns_time`	`app`, `location`
Server response / TTFB	`zscaler.zdx.server_response_time`	`app`
Device CPU / memory	`zscaler.zdx.device.cpu`, `.memory`	`device`, `location`
Packet loss / latency	`zscaler.zdx.network.packet_loss`, `.latency`	`path`, `isp`, `location`
ZPA connection setup	`zscaler.zpa.setup_time` — confirm the unit from your feed before converting	`app`, `connector`, `service_edge`
ZIA requests / errors	`zscaler.zia.request.count`, `.error.count`	`app`, `location`, `response_code`

Mind cardinality. `user` as a metric dimension will produce a metric with as many series as you have employees, which is a cost and a limit problem rather than an analytical one. Keep `user` on events and records; keep metrics dimensioned on `app`, `location`, `connector`, and similar bounded sets. FAQ-11 covers the cardinality mechanics.

> **Sources:** Understanding User Activity Log Fields (Zscaler) — *"The time in microseconds taken for processing in the central authority"* (`CAProcessingTime`); `ConnectionSetupTime` is described

> without a unit, Understanding the ZDX API (Zscaler Legacy APIs). **Derived:** the metric naming scheme and canonical dimension names are this entry's convention, not a Zscaler or Dynatrace standard — adopt or replace it wholesale, but do so once.

---

9. Dashboards, Alerting, and SLOs

With topology attached and metrics extracted, everything downstream is ordinary. Build it the way the rest of this corpus documents — this section says *what* to build; the named series say *how*.

9.1 Dashboards worth building

Aligned to audience and data plane. Field names depend on your normalization (§ 8), so treat these as content specifications.

Dashboard	Content
Executive experience overview	ZDX score by application and location; top degraded SaaS and private applications; open problems by application entity
ZPA private access operations	Sessions by application, user, location; connection-setup time p50 / p95 / p99; failures by application, connector, service edge; connector CPU, memory, active connections. Start from the ZPA extension's dashboards (§ 2.2)
ZIA SaaS operations	Top SaaS applications by volume; 4xx / 5xx by application; policy-induced latency and denies; throughput by application and location. Start from the ZIA extension's per-log-type dashboards (§ 2.1)
ZDX digital experience	Application score and trend; DNS / page-fetch / TTFB by application; endpoint CPU, memory, Wi-Fi; path latency and packet loss by ISP and location
Pipeline health	Records processed, metrics extracted, events created, records discarded by policy — the dashboard that tells you the integration itself is alive

That last one is not optional. A silently stalled LSS receiver or an expired ZDX credential presents as flat lines on the other four dashboards, which reads identically to "everything is fine."

9.2 Alerting

Raise alerts **against the entities from § 7**, never against raw log matches. An alert carrying an application entity, its connector, and its region is actionable; one carrying a log line is a research project. Every alert should answer four questions: which application is impacted, where, which layer is suspected, and what evidence supports it.

Alert	Entity context	Evidence
Private-app latency p95 breach	Application + connector + region	ZPA setup time, connector metrics
Connector saturation	App Connector + connector group	CPU, memory, active connections
Connector disconnected	App Connector	`lifetime[end]` older than 10 minutes — queried with a `from:` wider than the default 2 h window (§ 7)
SaaS error spike	Application	ZIA response codes, request counts
ZDX score degradation	Application + location / user group	ZDX score (with `-1` no-data values dropped), DNS / PFT / TTFB, device and network metrics
Feed stopped	The pipeline itself	Absence of expected metrics

With the ZPA extension, start from its alert templates: they ship disabled, and only its "Not Reporting" templates alert on absent data (§ 2.2).

`ALERT-01` covers the end-to-end architecture, `ALERT-02` the detection-mechanism decision, `ALERT-03` routing and cost, and `ALERT-04` the ServiceNow integration ladder. Where an external event-correlation platform is the destination, the topology work in § 7 is exactly what makes those payloads worth correlating.

9.3 SLOs

Extracted metrics are already the right shape for an SLI. Define the objective in Dynatrace — where the query can be validated against live topology — using `SLO-02` for SLI definition and `SLO-04` for burn-rate alerting. Where an external SLO tool consumes the data, it queries the same validated metrics through a service user; defining the SLI in Dynatrace first is what keeps the two consistent.

Candidate SLIs: ZDX score above threshold by application; private-application connection success rate; SaaS error rate by application.

> **Sources:** Zscaler Private Access (ZPA) extension (DT docs) — *"Alert templates are disabled by default."*

---

10. Synthetic Monitoring — When It Adds Anything

Start by assuming you do not need it. Where ZDX is licensed, it already provides active application and network-path measurement from real endpoints, which is most of what a synthetic test would tell you — and it measures where users actually are, which private locations only approximate.

Synthetic monitoring earns its place in three cases:

1. **No ZDX licence.** Synthetics become your only active experience measurement. This is a real gap-filler, though it measures a robot's experience, not a user's.
2. **You need to isolate Zscaler's contribution specifically.** Running the same journey through and around the Zscaler path is a controlled comparison ZDX does not offer.
3. **A journey ZDX does not cover.** A multi-step transaction through a private application, where per-step timing matters.

If you do proceed:

- **Deploy private synthetic locations where user workstations actually sit**, so the path traverses Zscaler the way a user's would. A synthetic in a data centre with a different egress path is measuring a different network.
- **Run browser and API tests against the critical SaaS applications** — the same ones already modeled as entities in § 7, so results correlate.
- **Test private applications through ZPA only where access is explicitly permitted.** This needs the ZPA administrator's agreement; a synthetic account with private-application access is a security decision.
- **Compare direct versus through-Zscaler paths deliberately**, not by default. Two tests per journey doubles the cost and only pays off when isolating Zscaler's contribution is the actual question.
- **Correlate results with the ZIA / ZPA logs and ZDX metrics** already in Grail — the correlation is the value, not the test in isolation.

The `SYNTH` series covers configuration; the decision above is the part specific to this integration.

---

11. Implementation Sequence

Phase	Objective	Key actions	Deliverable
1. Discovery	Know what feeds exist and who owns them	Inventory ZIA, ZPA, ZDX licensing (ZDX API access needs the Advanced Plan) and log types; identify existing pipeline and SIEM routing; confirm retention constraints and their owner	Data-source matrix with owners; a signed retention position
2. Route decision	One route per feed, from its transport	Apply § 4; choose Pattern A or B (§ 5); stand up the log shipper if Pattern A	An architecture that survives contact with ZPA
3. Ingestion	Data landing in Grail	Activate the ZIA extension and configure its Cloud NSS feeds; activate the ZPA extension, deploy its log shipper and configure one LSS receiver per log type with the extension's templates; build the ZDX poller	Validated datasets, one feed at a time
4. Normalize and extract	Signal, not volume	Rename to canonical dimensions; extract metrics and events; apply masking and hashing; **No storage assignment** for discard	Security-approved OpenPipeline rules
5. Topology	Entities, not strings	Build the § 7 node and edge processors for what the extensions do not cover; verify with `smartscapeNodes`	A verified topology model
6. Dashboards	Visibility	Build the five dashboards in § 9.1 — pipeline health first	Dashboards that filter on entities
7. Alerting and SLOs	Actionable output	Attach alerts to entities; define SLIs on extracted metrics; wire routing	An alert model with topology in the payload
8. Synthetics	Only if § 10 justifies it	Private locations on user paths; correlate with existing signals	Synthetic tests and baselines

Sequence notes. Do phase 1 before promising anything — ZDX licensing in particular decides what the integration can deliver (§ 3.4). Build phase 5 before phase 6, or you will rebuild the dashboards. Phase 3 is per-feed: finish one end to end before starting the next, because the second feed costs a fraction of the first once the pattern is established.

12. Administrator Checklist

Questions for the Zscaler administrator. Most integration delays trace to one of these being answered late.

Feeds and licensing

- Which **ZIA** log types are available — web, DNS, firewall, tunnel, DLP, audit? For the types the ZIA extension covers, can each Cloud NSS feed use the extension's exact JSON output format?
- Which **ZPA** log types are available — user activity, user status, App Connector status, App Connector metrics, browser access, audit, AppProtection?
- Is **ZDX licensed and enabled**, and is **API access** available? The ZDX API requires the **ZDX Advanced Plan**. (Decides whether experience monitoring is in scope at all.)

Routing and transport

- Are logs sent directly from Zscaler, or already routed through a pipeline product?
- If a pipeline exists, can it **multi-home** the selected streams to Dynatrace without changing the existing security output?
- For ZPA LSS: where can a **TCP listener** be placed that App Connectors can reach, and will TLS be used?
- Can each LSS receiver use the ZPA extension's exact log stream template — or, for a self-built pipeline, **JSON** rather than CSV or TSV?
- Can the ZPA log shipper run **highly available**? LSS does not resend logs (audit logs excepted) generated while the receiver is unreachable.

Security and data handling

- Which fields require **masking, hashing, exclusion, or drop-after-extract**?
- Who **signs off** on retaining raw records, and for how long?
- Which **user directory** backs the identity fields? (Determines whether hashing is meaningfully protective — § 6.)

Scope

- Which **applications, user groups, regions, and connector groups** are in scope?
- What are the **alert routing** requirements and destinations?
- Can **private synthetic locations** be placed behind the Zscaler path, and is synthetic access to private applications permitted?

13. Recommended Approach

If you have ZDX: pull it first. It answers the questions that generate the tickets, its volume is modest, and it is the fastest visible win. Add ZPA connector metrics next — it is the clearest infrastructure signal and small. Add ZIA and ZPA transaction logs last and in signal-first mode, because they are the largest and the ones most likely to become a storage problem if ingested before the extraction rules exist.

If you do not have ZDX: be explicit that this is an access, traffic, and infrastructure-health integration rather than experience monitoring, and scope the outcome accordingly. Lead with ZPA connector metrics and ZIA error rates, both of which are genuinely valuable and inexpensive. Decide separately whether synthetics (§ 10) are worth filling the experience gap with, or whether the ZDX licence is the better purchase.

In both cases:

Do	Instead of
Activate the ZIA and ZPA extensions for the feeds they cover	Hand-building parsers, metrics and topology they already supply
Route each product line by its own transport	One diagram arrow labelled "Zscaler → Dynatrace"
Extract metrics and events, discard raw by default	Retaining everything and deciding later
Use **No storage assignment**, not **Drop record**	Silently dropping records before extraction runs (FAQ-19 § 4.1)
Model applications, connectors, edges as entities	Filtering dashboards on log string fields
Hash user identity, keep it off metric dimensions	Retaining plaintext usernames, or exploding metric cardinality
Multi-home from an existing pipeline where one exists	Re-pointing a feed the SOC depends on
Build the pipeline-health dashboard early	Discovering a stalled feed via flat lines on a business dashboard

14. Common Gotchas

#	Gotcha	What to do
1	**Assuming ZIA and ZPA logs can replace ZDX.** They record transactions at the proxy; they contain no device health, path quality, or page-fetch timing	Scope experience monitoring on ZDX availability (§ 3.4)
2	**Drawing one arrow from Zscaler to Dynatrace.** ZIA pushes HTTPS, ZPA pushes raw TCP, ZDX telemetry is pull-only (its alerts can push by webhook)	Route per product line (§ 4)
3	**Expecting ZPA LSS to reach the ingest API.** LSS has no HTTPS output and no header support	Run a log shipper — Fluent Bit or BindPlane as the ZPA extension documents, an OTel Collector, or an existing pipeline; ActiveGate syslog only for a self-built pipeline (§ 4.1)
4	**One LSS configuration for all ZPA log types.** Each receiver configuration streams one log type	Plan a configuration per log type (§ 4.1)
5	**Leaving the LSS template on a default.** The ZPA extension cannot use the ZPA default template, which carries no log-type field; on a self-built pipeline, positional CSV parsing breaks silently when Zscaler adds a column	Use the extension's templates, or JSON for a self-built pipeline (§ 2.2, § 4.1)
6	**Polling the ZDX API aggressively — or too early.** Most report endpoints serve a two-hour window per request, data reaches the API an estimated 20 minutes after collection, and a score of `-1` means no data	Match cadence to the window and lag it; drop `-1` scores; build backfill as a loop (§ 4.2)
7	**No alert on feed absence.** An expired ZDX credential or stalled receiver looks exactly like "no problems"	Alert on absence of expected metrics; build the pipeline-health dashboard (§ 9.1)
8	**`Drop record` used for extract-then-discard.** It runs before every extractor	Use **No storage assignment** in the Bucket assignment stage (FAQ-19 § 4.1)
9	**Presenting ZPA timing as user experience.** Connection-setup time is a Zscaler-observed access-path proxy	Label it as such on dashboards; use ZDX for experience (§ 3.1)

#	Gotcha	What to do
10	`user` **as a metric dimension.** One series per employee	Keep `user` on events and records; dimension metrics on bounded sets (§ 8)
11	**Extracting the same node from two feeds.** Two sources of truth for one entity's fields	One defining feed per node type; others enrich by ID only (§ 7)
12	**Retaining raw into the default bucket** by omission	Dedicated bucket with its own permissions and retention (§ 6, FAQ-19 § 6)
13	**Building synthetics before checking ZDX.** Where ZDX is licensed it usually already answers the question, from real endpoints	Apply the § 10 test first
14	**Treating the field mapping in § 8 as a specification.** Names vary by feed type, template, and account	Validate against your own feed output before building
15	**Hand-building ZIA or ZPA when an extension covers it.** A parallel `zscaler.zia.*` or `zscaler.zpa.*` scheme and custom ZIA or ZPA nodes duplicate what the extensions supply	Activate the extensions (§ 2.1, § 2.2); keep custom work to ZDX and feed types they do not parse
16	**Reshaping a feed an extension parses.** A non-JSON ZIA output type, renamed fields, or a shipper that reformats ZPA records leaves the logs ingested but not parsed — no metrics, no topology, empty dashboards	Use each extension's exact format and forward ZPA records unmodified (§ 2.2, § 6)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.